
Hardness 5 out of 10

Part 1: Variables, Math & Logic (1–10)

1. **The BMI Calculator:** Create two variables: `weight_kg = 72` and `height_m = 1.75`. Calculate the Body Mass Index ($BMI = \text{weight} / \text{height}^2$). Write an if/elif/else block to print "Underweight" (below 18.5), "Normal" (18.5 to 24.9), or "Overweight" (25 and above).
2. **Swiggy Order Receipt:** You have `subtotal = 450`. Calculate a 5% GST and a flat 40 rupee delivery fee. Store the final sum in `total_bill` and print it using an f-string: "Your final bill is Rs. [amount]".
3. **Compound Assignment Tracker:** You have a bank balance `account = 5000`. Use shorthand assignment operators (`+=`, `-=`) to simulate: depositing 1500, withdrawing 300, and applying a 2x multiplier for a bonus. Print the final balance.
4. **Time Converter (Floor & Modulus):** You tracked a study session that lasted `total_minutes = 135`. Use floor division (`//`) to extract the hours, and the modulus operator (`%`) to extract the remaining minutes. Print the result as "2 Hours and 15 Minutes".
5. **Discount Logic:** Set `age = 24` and `is_student = True`. Write an if statement using the logical or operator. If the person is under 18 OR is a student, print "Discount Applied". Otherwise, print "Full Price".
6. **Olympic Year Checker:** Create a variable `year = 2026`. Write a condition using the modulus operator (`%`) to check if the year is perfectly divisible by 4. Print "Olympic Year" if true, and "Standard Year" if false.
7. **Temperature Safety Limits:** A PC processor is running at `temp = 82`. Chain your comparison operators to check if the temperature is strictly between 40 and 85 ($40 < \text{temp} < 85$). Print "Optimal Range" if true.
8. **Data Type Swap:** You have `raw_data = 12.89`. Cast it to an integer (which drops the decimal), convert that integer to a string, and concatenate it with the text " is the whole number".
9. **Boolean Failsafe:** Create a variable `user_input = ""`. Write an if statement checking the boolean evaluation of `user_input`. If it evaluates to False (because it's empty), print "Please enter a valid string."
10. **Scientific Yield:** Multiply the scientific floats `x = 1.5e3` (1500) and `y = 2e2` (200). Print the final mathematical result.

Part 2: String Manipulation & Parsing (11–20)

11. **Messy Data Cleanup:** You scrape a string from a website: `data = " fLuTtEr iS aWeSoMe "`. In a single line (by chaining methods), strip the whitespace and convert the entire string to Title Case.

12. **URL Username Extraction:** You have a profile link: `url = "https://github.com/tatai"`. Use the `string.split("/")` method. From the resulting list, use negative indexing to print only the username.
13. **Sentence Censor:** Take the string `feedback = "The battery is terrible and the screen is lagging."`. Use the `.replace()` method twice in a row to replace "terrible" with "****" and "lagging" with "****".
14. **Skipping Characters (Slicing):** You have a serial code: `code = "A1B2C3D4E5"`. Use string slicing with a step parameter `[::2]` to extract and print only the letters.
15. **Finding & Slicing:** You have a file path: `path = "C:/Projects/StudentRegistration.cs"`. Use `.find(".")` to locate the dot, and use string slicing from that index to the end to print just the file extension (".cs").
16. **Paragraph Splitter:** You have a multiline string with three sentences separated by the `\n` escape character. Use `.split("\n")` to turn it into a list, and print the length of that list to verify it has 3 items.
17. **Prefix/Suffix Verification:** You have `filename = "report_2026.pdf"`. Write an if statement using both `.startswith("report")` and `.endswith(".pdf")`. If both are true, print "Valid File".
18. **The Vowel Counter:** Create `text = "education"`. Set a `vowel_count = 0` variable. Write a for loop to check each letter. If the letter is in "aeiou", increment the counter by 1. Print the final count.
19. **Currency Cleaning:** You receive a string `price_tag = "Rs. 4500"`. Slice the string to remove the "Rs. " part, cast the remaining text into an integer, and multiply it by 2.
20. **Dynamic Padding:** Use the `.zfill()` method on the string "45" to make it exactly 5 characters long (e.g., "00045").

Part 3: Lists & Comprehensions (21–30)

21. **The Positive Filter:** You have a list of transactions: `ledger = [500, -20, 150, -100, 300]`. Create a new empty list. Loop through the ledger, and if the number is greater than 0, `.append()` it to the new list.
22. **Second Largest Number:** You have `scores = [85, 42, 99, 78, 91]`. Sort the list in ascending order, then use negative indexing to print the *second* highest score.
23. **List Comprehension (Math):** Write a single-line list comprehension that generates a list of the *squares* of numbers from 1 to 5 (e.g., `[1, 4, 9, 16, 25]`).
24. **List Comprehension (Condition):** Given `words = ["cat", "elephant", "dog", "panther"]`. Write a list comprehension that extracts only the words that have strictly more than 3 characters.
25. **Queue Management (FIFO):** You have an empty list `tasks = []`. Append "Task A", "Task B", and "Task C". Then, use `.pop(0)` to complete and remove the very first task. Print the remaining queue.
26. **String to List to String:** Take the string `"apple-banana-cherry"`. Use `.split("-")` to make it a list. Remove "banana" using `.remove()`. Then, use `.join()` to merge the remaining fruits back together with a comma.

27. **Matrix Extraction:** You have a nested list: `grid = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`. Use chained bracket indexing (e.g., `grid[row][col]`) to extract and print the number 5.
28. **In-Place List Modification:** You have `prices = [10, 20, 30]`. Write a for loop using `range(len(prices))` to directly modify the list by multiplying every value by 5.
29. **Parallel Printing:** You have two lists: `names = ["Amit", "Priya"]` and `marks = [85, 92]`. Write a for loop using `range(len(names))` to print "Amit scored 85", etc., by accessing both lists with the same index `i`.
30. **Common Element Check:** You have `list1 = [1, 2, 3]` and `list2 = [3, 4, 5]`. Use the `in` operator inside a loop to check if any item from `list1` exists in `list2`. If it does, print "Match Found".

Part 4: Tuples & Sets (31–40)

31. **Coordinate Unpacking:** You have a tuple `location = (22.5726, 88.3639)` (Kolkata coordinates). Unpack it into `lat` and `lng` variables. Print them.
32. **Tuple Modification Workaround:** You have a tuple `config = ("Dark Mode", "English", "v1.0")`. Cast it to a list, change "v1.0" to "v2.0", cast it back to a tuple, and print it.
33. **Unique Character Counter:** Given the string `word = "programming"`. Cast this string into a `set()`. Then print the `len()` of that set to find out exactly how many *unique* letters make up the word.
34. **Set Intersection (Friends):** You have `my_friends = {"Ravi", "Aman", "Sneha"}` and `your_friends = {"Sneha", "Karan", "Aman"}`. Use the intersection operator (`&`) to find and print only the mutual friends.
35. **Set Difference (Missing Stock):** Set `expected = {"Mouse", "Keyboard", "Monitor"}` and `actual = {"Mouse", "Monitor"}`. Use the difference operator (`-`) to figure out what item is missing from the actual stock.
36. **Symmetric Difference (Exclusives):** Using the friend sets from question 34, use the symmetric difference operator (`^`) to find all friends who are *not* mutual (i.e., they only exist in one set, not both).
37. **The Frozenset:** Create a frozenset from the list `[1, 2, 3]`. Try to use `.add(4)` on it. Write a comment above the line explaining why Python crashes when you run it.
38. **Mass Set Update:** Create an empty set `emails = set()`. Use the `.update()` method to simultaneously add a list of three strings and a tuple of two strings into the set at the same time.
39. **List Deduplication:** You have a messy list: `ids = [101, 102, 101, 103, 102]`. Convert it to a set to strip the duplicates, and then immediately convert it back into a list.
40. **Boolean & Zero Set Quirks:** Create a set `logic = {0, False, 1, True, 2}`. Print it and observe the output.

Part 5: Loops & Control Flow (41–50)

41. **The Bank Vault (While):** Set `pin = "1234"` and `attempts = 3`. Write a while loop that runs as long as `attempts > 0`. Inside the loop, print "Checking..." and decrease attempts by 1. When the loop finishes, print "Locked".
42. **Search and Break:** You have a list `logs = ["Info", "Info", "Error", "Info"]`. Loop through the list. If the item equals "Error", print "Critical Issue Found!" and use the break keyword to stop the loop immediately.
43. **Skip the Evens (Continue):** Write a for loop using `range(1, 11)`. Inside the loop, check if the number is even using the modulus operator (`num % 2 == 0`). If it is, use the continue keyword to skip it. Otherwise, print the number.
44. **The Asterisk Triangle:** Write a for loop from `range(1, 6)`. Inside the loop, print an asterisk "*" multiplied by the loop variable `i` (e.g., `print("*" * i)`) to generate a right-angle triangle pattern.
45. **Reverse Iteration (While):** Set `word = "Python"` and `index = len(word) - 1`. Write a while loop that runs as long as `index >= 0`. Print `word[index]` and decrease the index by 1 to print the string backward.
46. **Factorial Calculator (For):** Create a variable `total = 1`. Write a for loop using `range(1, 6)`. Inside the loop, multiply `total` by the loop variable `i` (`total *= i`). Print the final total (which will be the factorial of 5).
47. **Running Total Output:** You have a list `mileage = [45, 30, 60]`. Set `total_distance = 0`. Loop through the list, add the current item to the total, and print "Current distance: X" *inside* the loop.
48. **Simulated Elevator (While):** Set `current_floor = 1` and `target_floor = 5`. Write a while loop. If `current_floor < target_floor`, print the floor, then increment the floor by 1.
49. **Nested Loop Print:** Write a for loop that iterates over ["A", "B"]. Inside it, write another for loop that iterates over [1, 2]. Print the combinations (e.g., A1, A2, B1, B2).
50. **The Collatz Sequence (Basic):** Set `n = 10`. Write a while loop that runs as long as `n > 1`. Inside the loop, if `n` is even, floor divide it by 2 (`n //= 2`). If `n` is odd, multiply by 3 and add 1. Print `n` during each step.

Hardness of 6 out of 10

Part 1: Variables, Math & Logic (1–10)

- The Bill Splitter:** Set `total_bill = 1250` and `friends = 3`. Add a 10% tip to the total bill. Use floor division (`//`) to figure out how much each friend pays evenly without decimal coins, and use the modulus operator (`%`) to find the remaining change left over.
- Tax Bracket Logic:** Set `salary = 85000`. Write an `if/elif/else` block. If the salary is under 50,000, tax is 10%. If it is between 50,000 and 100,000, the tax is 20%. Otherwise, it's 30%. Calculate the exact tax amount and print it.
- Time Formatting:** You have `total_seconds = 7384`. Use floor division to find the total hours. Then use the modulus operator to find the remaining seconds, and use floor division on those remaining seconds to find the minutes. Print "H:M:S".
- Shorthand Bank Vault:** Set `balance = 1000`. Apply three transactions sequentially using shorthand operators (`+=`, `-=`, `*=`): deposit 500, withdraw 2000, and apply a 1.5x overdraft fee. If the final balance is less than 0, print "Account Overdrawn".
- Boolean Flagging:** Set `age = 20`, `has_license = True`, and `is_insured = False`. Write an `if` statement using the `and` operator to check all three. If age is ≥ 18 and both booleans are True, print "Can Drive". Else, print "Cannot Drive".
- Bitwise Masking:** Set `permissions = 5` (Binary 101). Read access is 4 (Binary 100). Use the bitwise AND operator (`&`) between `permissions` and 4. If the result is greater than 0, print "Read Access Granted".
- Data Type Pipeline:** Set `raw_data = "45.8"`. Cast it to a float. Add 10.2 to it. Cast that result to an integer (dropping the decimal). Finally, cast it to a string and concatenate it to print: "The final number is 56".
- Identity vs. Equality:** Create `list1 = [1, 2]` and `list2 = [1, 2]`. Set `list3 = list1`. Write an `if/elif` block to explicitly test and print whether `list1` is the exact same object in memory (`is`) as `list3`, and whether it shares the same values (`==`) as `list2`.
- Leap Year Logic:** Set `year = 2024`. Write an `if` statement using `and`, `or`, and `%` to check if it's a leap year. Rule: It must be divisible by 4, AND it cannot be divisible by 100 UNLESS it is also divisible by 400.
- Geometric Progression:** Set `start = 2` and `ratio = 3`. Use the exponentiation operator (`()`) to mathematically find the 5th term of the sequence without using a loop (Formula: $\text{Term} = \text{start} * \text{ratio}^{\{n-1\}}$).

Part 2: String Manipulation & Formatting (11–20)

- Domain Validator:** Set `email = "student@bibs.co.in"`. Use `.split("@")` to separate the name and domain. Write an `if` statement checking if the second part of the split list exactly equals "bibs.co.in".

12. **The Censor Tool:** Set `chat = "You are a noobri and a bot."`. Chain the `.replace()` method twice in one line to swap "noobri" with "***" and "bot" with "***". Print the safe chat.
13. **Dynamic Username:** Set `first = "Chittajit"` and `last = "Chakraborty"`. Use string slicing to extract the first 3 letters of both names. Convert them to lowercase and concatenate them with the number "99" to create and print a username.
14. **CSV Parser:** Set `row = " 101, Amit, 85 , MCA "`. Split it by the comma. Create a clean f-string that prints "ID: 101 | Name: Amit | Course: MCA", applying `.strip()` to each element as you insert it into the f-string.
15. **URL Formatting:** Set `title = "Python Sets and Tuples"`. Convert the entire string to lowercase, and use `.replace()` to swap every space " " with a hyphen "-" to generate a web-friendly URL slug.
16. **Vowel Masker:** Set `text = "programming"`. Write a for loop to iterate through the string. If the character is in "aeiou", print an asterisk * (using `end=""`). Otherwise, print the character itself.
17. **File Path Extractor:** Set `path = "/users/tatai/documents/report.pdf"`. Find the `.rfind("/")` (last occurrence of the slash) and `.find(".")`. Use those integer indexes to slice the string and print just the word "report".
18. **Palindrome Check:** Set `word = "Madam"`. Convert it to lowercase. Write an if statement checking if `word` equals `word[::-1]` (the reversed string). Print "Palindrome" or "Not a Palindrome".
19. **Prefix Sorter:** Set `filename = "IMG_9942.png"`. Write an if/elif/else block. If it `.startswith("IMG")`, print "Image". If it `.endswith(".pdf")`, print "Document". Else, print "Unknown".
20. **Quote Metrics:** Set `quote = "Learning Python is fun"`. Calculate the total number of words using `.split()`. Then, calculate the total characters *excluding* spaces by using `.replace(" ", "")` and finding the length. Print both numbers.

Part 3: Lists & Comprehensions (21–30)

21. **Queue Simulation:** Create `queue = ["Alice", "Bob", "Charlie"]`. "Dave" joins the back (append). "Alice" gets served and leaves (pop index 0). "Eve" cuts in line exactly at index 1 (insert). Print the final queue.
22. **List Merger & Sort:** Create `group1 = [45, 22, 12]` and `group2 = [11, 33, 50]`. Use `.extend()` to combine them. Sort the merged list in descending order, and use slicing `[:3]` to print only the top 3 numbers.
23. **Filtered Comprehension:** Set `prices = [100, 250, 40, 600, 15]`. Write a one-line list comprehension to create a `budget_items` list containing only the prices that are strictly ≤ 150 .
24. **Nested List Access:** Create `data = [{"A", 10}, {"B", 20}, {"C", 30}]`. Access the number 20 using bracket indexing `[][1]`, add 5 to it mathematically, and overwrite the original 20 in the list with the new value.

25. **Word Lengths List:** Create words = ["Python", "is", "awesome"] and an empty list called lengths. Use a for loop to iterate through words, calculate the len() of each word, and .append() that number to the lengths list.
26. **Parallel Lists:** Set items = ["Pen", "Notebook"] and qty = [5, 2]. Write a for loop using range(len(items)) so you can print a combined string: "We have 5 of Pen", "We have 2 of Notebook".
27. **Remove Duplicates Loop:** Create nums = [1, 2, 2, 3, 4, 4, 5] and an empty unique_nums list. Write a for loop. If the current number not in unique_nums, append it. Print the clean list.
28. **In-place Math Loop:** Set values = [10, 20, 30]. Use a for loop with range(len(values)) to target the list indexes. Divide every number by 2 and overwrite the original index with the new float.
29. **Ternary Comprehension:** Set marks = [45, 82, 30, 90]. Write a list comprehension using if/else logic to generate a new list that says "Pass" if the mark is ≥ 40 , else "Fail".
30. **Split & Combine:** Set lst = [1, 2, 3, 4, 5, 6]. Slice the first 3 items into part1 and the last 3 items into part2. Combine them backward into a new list: part2 + part1.

Part 4: Tuples, Sets, and Memory (31–40)

31. **Tuple Unpacking with Rest:** Set scores = (99, 85, 92, 78, 65). Unpack this tuple into variables first, second, and *rest. Print the first variable, and print the rest variable (notice it becomes a list).
32. **Tuple Immutability Trick:** Create config = ("192.168.1.1", 8080, "HTTP"). You need to update the port. Cast the tuple to a list, change 8080 to 443, cast it back to a tuple, and print it.
33. **Set De-duplication Flow:** Create emails = ["a@a.com", "b@b.com", "a@a.com"]. Cast it to a set to automatically destroy duplicates. Cast it back to a list, apply .sort(), and print the clean list.
34. **Set Intersection Count:** Set math_class = {"A", "B", "C"} and science_class = {"B", "C", "D"}. Use the & operator to find the students enrolled in both, and use len() to print the total count of dual-enrolled students.
35. **Set Difference:** Set required = {"Form6", "Aadhar", "Photo"} and provided = {"Aadhar"}. Use the - operator to find what is missing. Print an f-string: "Please submit: {missing_items}".
36. **Symmetric Difference:** Set group_a = {1, 2, 3} and group_b = {3, 4, 5}. Use the ^ operator to find the numbers that are unique to each group (not shared by both).
37. **Mutable Tuple Elements:** Create tup = (1, 2, [3, 4]). Access the list nested inside the tuple using indexing [2], and use .append(5) to add a number to it. Print the tuple to see that lists inside tuples *can* be modified.
38. **Mass Set Update:** Create tech = {"Python"}. Use the .update() method to simultaneously merge a list ["Dart", "Flutter"] and a tuple ("C#", "Java") into the set.
39. **Safe Removal (discard):** Create cart = {"Apple", "Banana"}. Use .discard("Cherry") to attempt removing a non-existent item safely. Then, use .remove("Apple"). Print the final set.

40. **Boolean/Int Set Collision:** Create `quirk = {0, False, 1, True, 2}`. Print the set and write a one-line comment in your code explaining exactly why the output looks the way it does.

Part 5: Loops & Control Flow (41–50)

41. **While Loop Validation:** Set `target_len = 8` and `password = "pass"`. Write a `while len(password) < target_len` loop. Inside, append `"x"` to the string and print `"Too short, adding x..."`. Print the final valid password.

42. **For Loop Factorial:** Set `num = 5` and `fact = 1`. Write a for loop using `range(1, num + 1)`. Multiply `fact *= i` on each iteration. Print the final calculated factorial.

43. **The Break Condition:** Set `temperatures = [35, 36, 40, 32]`. Loop through the list. If `temp >= 40`, print `"Overheating!"` and break the loop instantly. Else, print `"Normal"`.

44. **The Continue Condition:** Set `transactions = [200, -50, 300, -10]` and `total = 0`. Loop through them. If the transaction is `< 0`, use `continue` to skip it. Else, add it to `total`. Print the sum of positive transactions.

45. **Iterating String Backwards:** Set `word = "Python"`. Set an index variable to `len(word) - 1`. Write a `while index >= 0` loop. Print `word[index]` and decrement the index by 1 each loop to print backward.

46. **Nested Loop Grid:** Write an outer loop for `row` in `range(3)` and an inner loop for `col` in `range(3)`. Inside the inner loop, print the coordinates formatted as `(row, col)`.

47. **Multiplication Table:** Set `n = 7`. Use a for loop with `range(1, 11)`. Use an f-string to print the mathematical table exactly like this: `"7 x 1 = 7"`, `"7 x 2 = 14"`, etc.

48. **Running Average:** Set `scores = [80, 90, 100]` and `running_sum = 0`. Loop through the list using `range(len(scores))`. Add the score to the sum, calculate the average at that specific index (`running_sum / (i + 1)`), and print it.

49. **Elevator Simulation:** Set `current = 0` and `target = 4`. Write a `while current < target` loop. Print `"Passing floor X"`, then increment `current` by 1. When the loop finishes, print `"Ding! Arrived."`

50. **Digit Summation:** Set `num = 456` and `total_sum = 0`. Cast `num` to a string. Loop over the characters in the string, cast each character back to an integer, add it to `total_sum`, and print the final mathematical sum (which should be 15).